Mandate — the complete field guide

Team 10 · Raingentic Commerce Hackathon NYC · 8–9 August 2026 Om Thakur and Princy Doshi

This document explains the whole project in plain words: what the event is, what every word means, what we built, what every page does, what happens when you press each button, what the code is, what is real and what is not, and what to go and ask people.

It assumes you know nothing. Read it top to bottom once and you will be able to explain this project to anybody in the room.

---

1. Where you are

1.1 The event

The **Raingentic Commerce Hackathon**, New York, 8–9 August 2026. The theme is *agentic commerce* — AI agents that spend real money on behalf of a business.

You are **Team 10**. You have a paper credentials sheet with a team ID, a user ID, an API key and a collateral contract ID.

1.2 The three organisations, and which is which

People will mention these by name and assume you know the difference.

Rain — the main sponsor. They issue payment cards funded by stablecoins. The important detail: they are a Visa and Mastercard **Principal Member**, which means they issue cards directly rather than reselling another bank's licence. Most fintechs cannot say that.

Their newest product is the **Agent Control Layer**: programmatic limits on an AI agent's card — how much, which merchants, how often. Their own published wording is that these are enforced "*at card issuance and transfer initiation rather than applied after the fact.*" Remember that phrase. Our entire pitch is built on it.

Three of the five judges work at Rain.

Monad — a blockchain, compatible with Ethereum tooling, whose selling point is speed and very low transaction cost. That cheapness is the point for us: it makes it affordable to write small records onto the chain routinely. They offer a bounty for projects where the chain genuinely matters rather than being decoration.

Encode — the developer community organising the weekend. They run hackathons and educational programmes. Not a technology sponsor; the host.

1.3 The judges

Five people. Three from Rain — a product lead, an engineer who works on high-throughput transactional systems, and a data engineer who has personally won blockchain hackathons. That last one matters: he will recognise a staged demo instantly. The other two are an engineering manager at Cursor and the AI engineering lead at Monad.

This shaped real decisions in the build, which section 5 explains.

---

2. Every word you will hear

None of these are hard ideas. They just have names that sound like they are.

Word	What it actually means
Agent	Software doing a job by itself, with no person clicking each step. Here, one that buys things.
Purchase order (PO)	A company's formal record: "we are buying this, from them, at this price, this quantity." Effectively the receipt written *before* the purchase.
Quote	A supplier's price offer. It has an expiry, because a price is only promised for so long.
Line item	One row on an order: the specific thing being bought.
SKU	The product code for that thing. `VO-CHAIR-M4` is a SKU.
Vendor / supplier	The company being paid.
Cost centre	Which department's budget the money comes from. Engineering, Facilities, Operations, Marketing.
Scoped card	A card that only works for one specific purchase — this supplier, this amount, expiring soon. As opposed to an ordinary card that works anywhere.
Virtual card	A card that exists only as numbers. No plastic. Created and destroyed in software.
Issuance	The moment a card is created. The single most important word in our pitch.
Settle	The moment the money actually moves.
Revoke	Turning a card off so it cannot be used again.
Idempotency	Doing something twice has the same result as doing it once. In payments: a retry must not pay twice.
Append-only log	A record you can add to but never edit or delete. What makes an audit trail worth trusting.
Snapshot	A frozen copy of what the records said at one moment, stored forever with the decision.
Replay	Re-running past decisions under a changed rule, to see what would have been different.
Delegated authority	How much someone is trusted to spend without asking. Every company has this for staff. We give it to agents.
Escalation	Sending something to a human instead of deciding it automatically.
Structuring	Deliberately splitting one big payment into several small ones to stay under an approval limit. A real fraud pattern, illegal in banking.
Three-way match	The classic accounting control: the order, the delivery and the invoice must all agree before payment.
Stablecoin	A cryptocurrency pegged to a normal currency, so one unit stays worth roughly one dollar. What funds Rain's cards.
RUSD	The specific stablecoin in Rain's system.
Collateral contract	The pot of money backing your cards. Without one funded, an issued card has no spending power.
Testnet	A practice version of a blockchain. Real software, worthless money.

Word	What it actually means
Anchor	Writing a small fingerprint of some data onto a blockchain so you can later prove it existed at that time and has not changed.
Hash	A short fingerprint of a piece of data. Change one character of the data and the fingerprint changes completely.
Deterministic	Same inputs always produce the same output. No randomness, no AI guessing.
Pure function	Code that only looks at what it was handed, touches nothing else, and changes nothing. Easy to trust and easy to test.

---

3. The problem

An AI agent needs to buy something. Rain can give it a card, and Rain can control **how much** it spends and **where** it spends it.

But no card control can tell you **why** the agent is spending.

Here is the concrete failure. A card is locked to "Pallas Logistics, up to $1,000". The agent buys the **wrong item** from Pallas Logistics for $960.

- Right supplier — the card control is satisfied.
- Right amount — the card control is satisfied.
- Completely wrong purchase — the card control cannot see this at all.

The money is gone. Nobody finds out until month end, if ever.

That is the gap. **The card knows the shape of the spending. It does not know the reason.**

---

4. What we built

Mandate checks the reason before the card exists.

The agent declares what it is buying and why. Deterministic code compares that declaration against the company's real records. Only if it holds up does Rain issue a card scoped to exactly that purchase.

If it does not hold up, **no card is ever created.**

That is a stronger claim than blocking a payment. There is no card to cancel, dispute or claw back, because none was ever born.

The sentence to say out loud

> Rain bounds **how much** an agent spends and **where**. Mandate bounds **why** — one step earlier, before the card exists.

Who it is for

This is a product for **Rain's customers**, not for Rain staff. The user is the finance function — a controller or the CFO's office — at a company that wants to let agents buy things. They own spending policy and they carry the blame when money goes out wrong.

Today that person vetoes "give the AI a card" outright. Mandate removes the veto. That is also why Rain should care: it makes their Agent Control Layer sellable into companies that would otherwise say no.

---

5. The four ideas that make it defensible

5.1 No AI makes the decision

The agent proposes. **Eleven plain functions decide.** There is no model in the decision path, no I/O, and no reading of the system clock.

This is deliberate and it is the foundation of everything else. Because the checks are deterministic, re-running history means something: the same inputs always give the same answer, so any difference in a replay can only have come from the rule change. If a model sat in that path, a replay would prove nothing.

An AI *does* run — it writes the suppliers' negotiation dialogue — but it runs **upstream** of the checks and can only touch wording, never a price or an outcome.

5.2 The rules are data, not code

Every threshold is a row in a table that a finance team can edit, not a number buried in software needing a developer and a deployment.

Editing a rule does not overwrite the old one — it writes the **next version**. Old versions stay forever. And a version does nothing until **a second person activates it**; the author cannot approve their own change. That is called segregation of duties, and it is the answer to the obvious criticism: "*you can just change the rules, so your audit proves nothing.*"

5.3 Three outcomes, not two

Outcome	Meaning
Approved	Every check passed. A card was issued for exactly this purchase, then retired.
Refused	A check failed. No card was ever created. Nothing to cancel.
Held	Nothing is wrong — it is simply above the agent's authority. No card exists until a named person releases it.

The distinction between *refused* and *held* is the important one. A purchase that is **wrong** and a purchase that is merely **large** need different answers. Collapse them and you either block legitimate spending or wave through the thing you most wanted a human to look at.

5.4 The decision stores a snapshot, not a pointer

When a decision is made, we save a **frozen copy** of what the records said at that moment, not a link to records that will keep changing.

This sounds like a detail. It is what makes replay honest. If you replayed a link six months later you would be re-judging today's facts, not the facts that were true when the decision was made.

---

6. Every page, and what it is for

There are six pages. One shared navigation bar appears on all of them, in this order.

6.1 Workspace — `/workspace`

The customer-facing home. What a finance person would see day to day.

6.2 Console — `/` (the main demo screen)

The operations view, and where you will spend the demo. It has three numbered sections.

Section 1 — How it works. A diagram of the pipeline that lights up as a real run passes through it. Underneath, once you run something, a second **vertical view** shows the same run falling top to bottom, so a refusal visibly stops partway down.

Section 2 — Run it. The task buttons, the decision log, and the department budgets.

Section 3 — Audit it. Two tabs:

- **Provenance** — one decision explained in full: what was bought, what happened, and all eleven checks with the reasoning. Plus a *Download PDF* button that produces a real receipt.
- **Policy** — the rule editor and the replay tool.

At the top right, three status badges that always tell the truth:

Badge	Meaning
`postgres` / `in-memory`	Whether decisions are saved to a real database or will vanish on restart.
`rain live` / `cards simulated` / `rain not connected`	Whether cards are genuinely being issued by Rain.
`policy v1`	Which version of the rules is deciding.

6.3 Catalogue — `/catalog`

Eight products with drawn illustrations, real prices and quantity pickers. Two kinds:

- **Negotiated** — no quote exists yet, so suppliers compete and the winner becomes the order. Pick a quantity and the negotiation genuinely runs.
- **On contract** — a quote already exists, so the declared total must match it. Change the quantity and watch it get refused. The card tells you before you click.

Buying here posts to the **same endpoints** the console uses. There is no catalogue-only path around the checks.

6.4 System design — `/architecture`

For a technical judge. Contains the full architecture diagram, a step-by-step walkthrough, the technology stack with the reason for each choice, all eleven checks explained, why Rain specifically, how it maps to the submission tracks, and what we deliberately did **not** build.

6.5 Agents — `/agents`

The five buyers, what each one handles, why each is named what it is, and live statistics counted from the real decision log.

6.6 Deck — `/presentation`

The pitch as a page rather than a PDF, so a slide is one click from the screen that proves it. **Arrow keys** to move, **F** for fullscreen.

---

7. The five agents

Every purchase is declared by one of these. The name is cosmetic; the id is what the system keys on. Three are named for the hosts, two for the history of financial control.

Name	Id	Handles	Named for
Rae	`facilities-01`	Facility purchases	**Rain**
Mona	`procurement-01`	Freight and logistics	**Monad**
Cody	`cloud-compute`	GPU compute contracts	**Encode** (en*CODE*)
Prue	`procurement-02`	Capital purchases over $25,000	The **prudence principle** in accounting
Luca	`office-supplies`	Office supply negotiations	**Luca Pacioli**, who codified double-entry bookkeeping in 1494

There is also **"a person"** (`catalog`), used when a human buys from the catalogue by hand. It goes through identical checks — the rules do not care who is asking.

Why five? Not an arbitrary number: **one agent per distinct verification story.** Two negotiation paths, two different kinds of refusal, and one escalation.

---

8. What happens when you press a button

Every purchase, from any page, goes through the same pipeline.

8.1 The pipeline, stage by stage

#	Stage	What happens
1	**NEGOTIATE**	Suppliers compete. Each concedes according to its own fixed strategy and will not go below its floor. Cheapest qualifying bid wins. Only on negotiated purchases.
2	**PROPOSE**	The agent declares the purchase order: supplier, item, unit price, quantity, expiry.
3	**VERIFY**	A snapshot of the records is taken and frozen. Eleven checks run against it.
4a	**REFUSE**	Any check failed. No card is created. The reason is written down. **Stops here.**
4b	**HOLD**	Everything passed but it is above authority. No card exists. Waits for a person. **Stops here.**
4c	**ISSUE**	Everything passed. Rain issues a card scoped to exactly this amount.
5	**SETTLE**	The purchase happens. The department budget is charged and the order marked fulfilled.
6	**REVOKE**	The card is retired. It existed for exactly this purchase and no longer.
7	**RECORD**	Everything is appended to the log: the order, the snapshot, all eleven verdicts, the rule version, the card, the outcome. Never edited afterwards.

8.2 The eleven checks, in plain English

Each one is a yes/no question. The screen shows them exactly this way.

#	The question	Why it exists
1	Is this order on the books?	Catches an agent inventing an order nobody raised.
2	Is it still open and unpaid?	Stops paying twice for the same thing.
3	Does the amount match the quote?	Catches a price that drifted from what was agreed.
4	Is it the right supplier **and** item?	The one no card control can see. Right vendor, right total, wrong thing.
5	Is there budget left?	Checked against the real remaining balance, not a cached number.
6	Has a card already been issued?	A retry must not create a second card.
7	Is it inside spending authority?	Over $25,000 a person must release it. Held, not refused.
8	Is it a big purchase split up?	Stops an agent buying twice just under the limit to dodge check 7.
9	Inside this agent's own limit?	Luca is trusted to $2,000, Prue to $50,000. Not everyone is equal.
10	Have we paid this supplier before?	Invoice fraud almost always arrives as a payee nobody has paid. Held, not refused.
11	Is it buying too fast?	A looping agent makes purchases that are each perfect. Only the rate is wrong.

Checks 8 to 11 were added last, and 8 exists specifically because check 7 has an obvious weakness: buy twice, just under the limit each time.

8.3 What each demo button does

Button	What it does	Result
Restock office supplies	Four suppliers bid, one counter-offer round	Approved, card issued and retired
Restock office supplies *(again)*	Same task, same order line	**Refused** — duplicate
Provision GPU compute	Three suppliers, tighter market	Approved
Buy replacement chairs	Right item and price, supplier who never quoted	**Refused** — check 4
Book EU freight lane	Right supplier and total, wrong line item	**Refused** — check 4
Order conveyor line	Correct in every way, but $43,500	**Held** for a person
Write your own purchase order	A form, prefilled with something that passes	Whatever you make it
Reset demo	Clears this session's rows, keeps seeded history	—
Replay against history	Re-judges every recorded decision under your edited rule	A count of what would flip

---

9. The technology, and why each piece

Layer	What we used	Why it had to be this
Framework	Next.js 14, React 18, TypeScript	One deployable for the screen and the API.
Styling	Tailwind CSS	Colours sampled from Rain's and Monad's real stylesheets, not guessed.
Fonts	Fraunces, Inter, Roboto Mono	Rain's site uses a proprietary serif we cannot licence, so Fraunces stands in. Monad genuinely uses Roboto Mono — that one is exact.
Payments	Rain issuing API	Cards are the enforcement point. Auth is an `api-key` header, confirmed against the live sandbox.
Chain	Monad testnet, via viem	Each rule version's fingerprint is anchored so policy cannot be rewritten after the fact.
Database	Postgres on Neon	The append-only decision log.
Verification	Plain TypeScript, no libraries	Eleven pure functions. No model, no I/O, no clock — which is what makes replay meaningful.
Money	Integer cents everywhere	No decimals near a currency value, so a tolerance check cannot be beaten by rounding.
Negotiation	Deterministic strategy engine	Same task always produces the same winner, so the result is checkable.
AI	Groq, `llama-3.1-8b-instant`	Writes the suppliers' dialogue. Capped at 40 words, three second timeout, silent fallback.
Documents	jsPDF	A downloadable receipt per decision, loaded only when clicked.
Tests	52 automated tests	Every check tested on both sides of its boundary.

One bug worth knowing about

The database driver talks over HTTP, and Next.js caches HTTP calls by default. Without disabling that cache, the first read is memorised and replayed forever: writes land in the database, but the screen shows a decision count that never moves. No error appears anywhere. It is fixed, and it is written down because it would have broken the deployed demo silently.

---

10. What is real and what is not

Say this before anyone asks. Volunteering it is worth far more than being caught by it.

Part	State
The eleven checks	**Real**, 52 tests
Negotiation between suppliers	**Real**, deterministic
Append-only decision log	**Real**, in Postgres
Replay across history	**Real**
Human approval and release	**Real**
Two-person rule changes	**Real**
Connection to Rain's API	**Real**, authenticated
AI writing supplier dialogue	**Real**, Groq
Card issuance	**Simulated**
Monad anchoring	**Built, switched off**

Why the cards are simulated — the good version of the answer

It is not that the integration fails. Rain **does** create a card. But it currently ignores the spend limit we send, handing back a card with **no limit and a 2031 expiry**.

Our entire claim is *a card scoped to exactly this purchase*. So the code refuses to present an unscoped card as a scoped one, falls back to a simulated card, and labels it simulated everywhere — including the badge at the top of the screen, which reads `cards simulated` rather than `rain live`.

We also switched real issuance **off** by default, because each attempt leaves behind a live, unscoped card that cannot yet be deactivated, in exchange for nothing.

> **Choosing not to overclaim is the product.** Say exactly that. It turns the one obvious weakness into the clearest possible demonstration of the thesis.

The headline demo moment is a purchase that was **stopped** — and a refusal never needed a card to exist.

---

11. The demo, in the order to click it

1. Run "Restock office supplies." Four suppliers bid, one wins, the winning price becomes the order. The pipeline lights up: verified, issued, settled, retired.

2. Run the exact same task again. *(the headline)* Refused as a duplicate.

> "The record the first run wrote is what refuses the second. I couldn't fake this if I wanted to."

3. Run "Book EU freight lane." Right supplier, right total, wrong line item. Refused.

> "No card control on earth catches this one."

4. Run "Order conveyor line." Nothing wrong with it — it is $43,500, so it is held for a person, not refused. And no card exists while it waits, so approving is what *creates* it, not a review of something already live.

5. Policy tab → change a threshold → "Replay against history." It re-judges every recorded decision and tells you how many would flip. This proves the rules are data a finance team owns, not code needing a deploy.

The two questions you must have answers for

"Is an AI making these decisions?"

> No, and deliberately. The agent proposes; eleven pure functions decide. That is exactly why replaying history means anything — same inputs, same answer, so a difference can only have come from the rule change. An AI does run, writing the suppliers' dialogue, but it sits upstream of the checks and can only change wording. Turn it off and every number is identical.

"Where is the human?"

> Above a limit you set, exactly like a delegation-of-authority matrix for staff. Nobody gives a person unlimited spending authority either.

---

12. Questions to go and ask

You are trying to learn one thing: **is this differentiated, or does it already exist?** Ask questions that could genuinely come back "no", or you will only collect politeness.

12.1 Rain engineers — ask first, they leave when the venue closes

Collateral. "*Our user shows approved and active, but zero collateral contracts, and the contract ID on our sheet returns 403. Does it need attaching to the user, or funding with RUSD first?*" → This is the only thing between simulated and real cards.

The spend limit. "*We send a spend limit inside* `configuration` *and the card comes back with only* `{currency: usd}`*. What is the correct field name to make a limit stick?*" → Answer goes straight into the code. No rewrite.

Deactivating a card. "`PATCH /issuing/cards/{id}` *exists, but* `status: inactive` *returns 400. What value deactivates a card?*" → Answer goes into one environment variable. Revoke then works with no code change.

12.2 Judges, mentors, Rain product people

The one that matters most. "*What would you compare this to? Does it already exist?*" If three people name the same product, that is your answer and you pivot.

Test the core claim. "*If you were the CFO, would this change your mind about giving an agent a company card? If not, what would?*"

Invite the attack. "*What is the first thing you would try to break about this?*" People enjoy answering this and it is where the real feedback lives.

Check the differentiator is one. "*We kept the AI out of the decision path so history can be replayed. Is that interesting, or obvious?*"

For Rain specifically. "*Does this compete with your Agent Control Layer, or sit on top of it?*" If they say compete, reposition tonight.

12.3 Other builders

The thirty-second test. Explain it in thirty seconds, then ask them to say it back. If they cannot, the pitch is wrong — not their listening.

12.4 Be ready for the hardest question

Somebody will ask how this differs from Coupa, Ramp or Airbase — existing procurement and spend-management software worth billions.

> Those approve invoices *after* the money moved, or control a card that already exists. We bind the control to the moment of **issuance**, so a failed check produces no instrument at all. And it is built for agents rather than for people filling in forms.

Have that sentence ready. It is a genuine difference, but only if you can say it quickly.

---

13. Still outstanding

Item	Blocked on
Real card issuance	The three Rain answers above
Monad anchoring going live	A testnet RPC URL and a funded testnet key
Deployment	Putting `DATABASE_URL` into Vercel's environment variables — without it the deployed site silently loses the decision log on every cold start, which breaks replay and the duplicate refusal
Submission	The Encode platform. Pitching order follows submission order

---

Repository: github.com/theomthakur/raingentic-team10